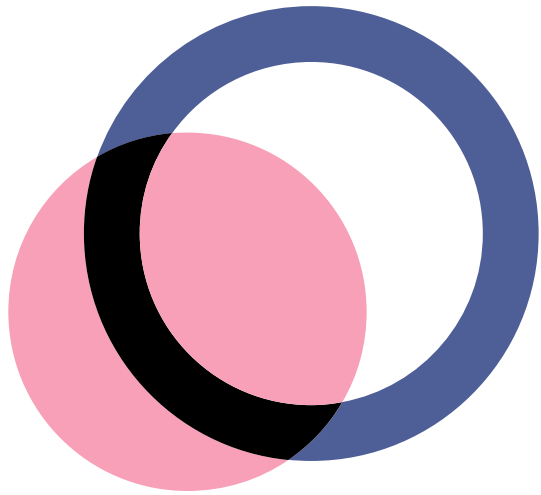
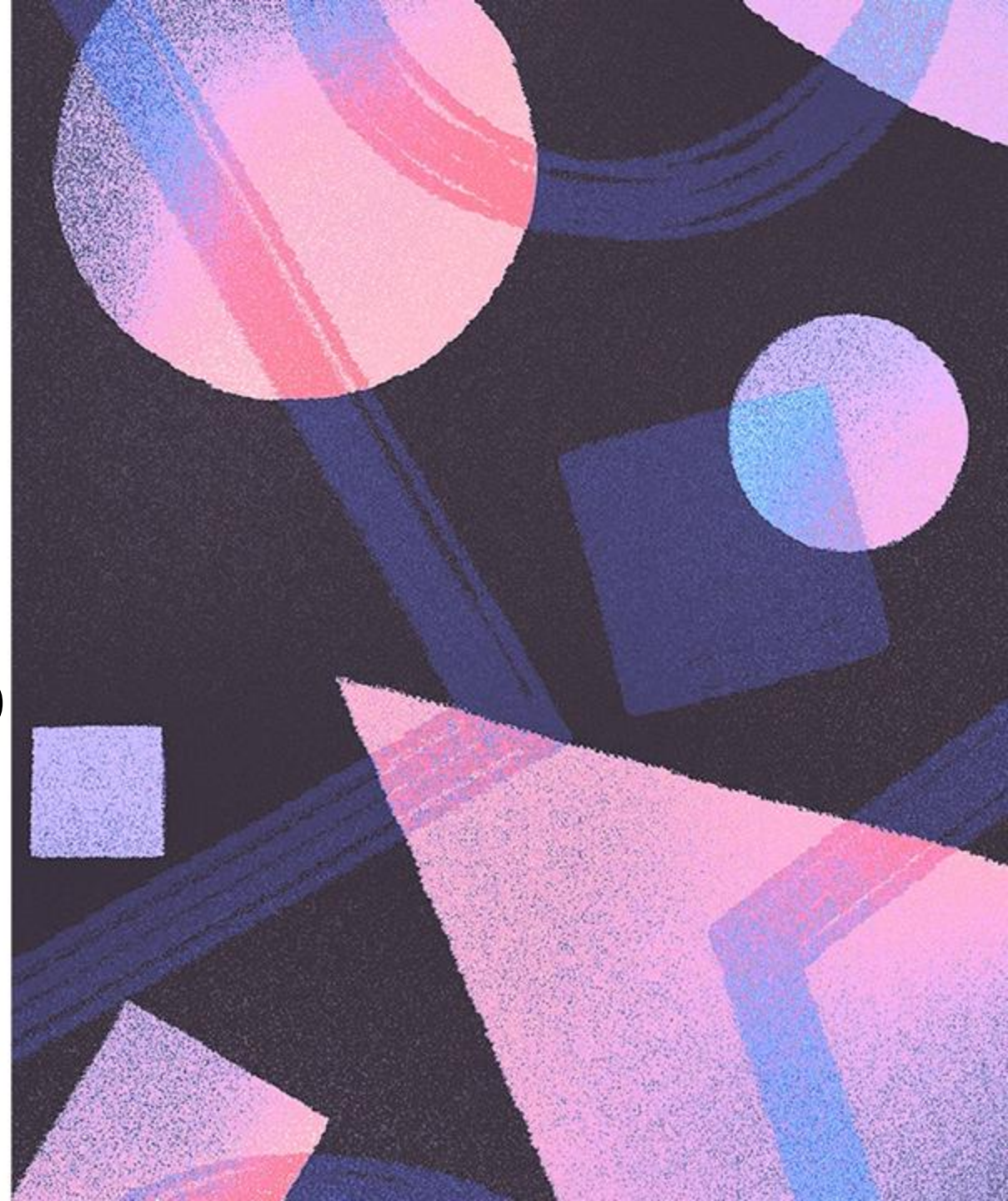




Design Patterns Lab

Lecture 9

> Decorator Pattern



Decorator Pattern

- Part of Structural Design Patterns
- Purpose: add behavior dynamically to objects
- Core Idea: wrap objects with additional layers
- Avoids:
 - modifying original classes
 - inheritance explosion
 - Adds responsibilities externally

Backend Use Cases

Decorator Pattern is heavily used in:

- API logging systems
- Authentication filters
- Request validation
- Response caching

Core Components

- Component: Base interface
- Concrete Component: Original implementation
- Decorator: Abstract wrapper
- Concrete Decorator: Adds extra behavior
- Core Flow:
Decorator → Service → Execution

Backend Scenario

- Backend system contains: basic order service
- Core functionality: place orders
- New business requirements:
 - log requests
 - measure execution time
- Requirement: avoid modifying original service
- Solution: apply decorators around the service

Key Observation

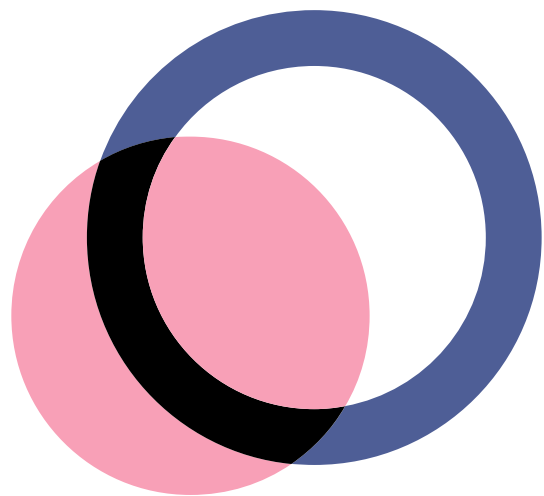
- Original service remains unchanged
- Features added externally
- Behavior extended dynamically
- Strongly follows:
 - Open/Closed Principle

Advantages & Disadvantages

- Advantages
 - No modification of existing classes
 - Flexible runtime behavior
 - Reusable decorators
 - Clean separation of concerns
 - Scalable architecture
- Disadvantages
 - Many small classes
 - Debugging becomes harder
 - Execution order matters

Activity: API Request Processing Pipeline

- A backend API processes client requests through multiple optional features like Logging, Authentication, Caching, and Rate Limiting
- The core request handler should remain unchanged while extra behaviors are added dynamically
- Design decorators for each feature and wrap them around the main request service
- The system should support combining multiple decorators in different orders at runtime



**THANK
YOU**

